

Driver App – Detailed Functional Requirements Document

Developer-ready functional specification for a production Driver / Fleet Management application

1. Purpose

- Digitally maintain the driver's duty, assigned vehicle, trip, odometer, kilometres, fuel, expenses, advances, payments, and documents in one system.
- Core record chain: Driver → Vehicle → Duty → Odometer → KM → Fuel → Fuel Photos → Expenses → Receipts → Advance → Payments → Documents → Trip.

2. Driver Profile & Vehicle

- Driver fields: Driver Name, Driver ID, Mobile Number, Licence Number, Licence Validity, Profile Photo, Status.
- Vehicle fields: Vehicle Number, Vehicle Type, Model, Vehicle ID, assigned trip, and trip status.
- Driver profile must clearly show the currently assigned vehicle.

3. Login & Authentication

- Driver login using phone number and 4-digit PIN.
- Production implementation must use secure authentication, session/token management, role-based permissions, and protected APIs.

4. Home Dashboard

- Show driver name, assigned vehicle, current trip, duty status, odometer, daily KM, fuel, expenses, and advance/wallet balance.
- Quick actions: Add Fuel, Add Expense, Trip Documents, Advance Request, Digital ID, Report Accident/SOS.
- Bottom navigation: Home, Wallet, SOS, Docs, Profile.

5. Daily Duty & Odometer

- Start Duty: automatically capture date/time, starting odometer, and start-odometer photo; GPS may be captured.
- End Duty: automatically capture end time, ending odometer, end-odometer photo, and optional remarks.
- Total KM = Ending Odometer – Starting Odometer.
- Validation: a new odometer reading cannot be lower than the previous valid reading; end odometer cannot be below start odometer.
- Store duty history with date, start/end time, start/end odometer, total KM, photos, and status.

6. Fuel Entry & Fuel History

- Required fields: date/time, odometer, fuel litres, total cost, and fuel pump/meter photo.
- Optional fields: fuel station, fuel type, receipt, remarks, and location.
- Price per litre = Total Cost ÷ Litres.
- Fuel history must preserve all entered values and associated photos.

- Validate odometer progression and warn when the entered quantity appears inconsistent with configured tank capacity.
- Fuel efficiency can be calculated as $\text{distance} \div \text{fuel consumed}$.

7. Expense Management

- Categories: Toll, Food, Parking, Repair, Loading/Unloading, Vehicle Maintenance, Other.
- Fields: category, amount, date/time, note, receipt photo, location, and linked trip.
- Receipt photo is required for expense submission in the prototype flow.
- Approval states: Pending, Approved, Rejected, Paid/Reimbursed.
- Expense history and daily summaries must be available.

8. Advance Request & Settlement

- Request fields: amount requested, reason, request date/time, and linked trip.
- Statuses: Pending, Approved, Rejected, Paid, Settled.
- Show how much was requested, approved, paid, and later settled.
- Reconciliation example: ■10,000 advance – ■7,500 approved expenses = ■2,500 remaining.

9. Wallet / Petty Cash

- Maintain opening balance, advances received, expenses, fuel, adjustments, and remaining balance.
- Provide a complete transaction ledger with transaction type, amount, date/time, reference, and running balance.

10. Trip Linking

- Link duty logs, odometer readings, fuel entries, expenses, advances, and documents to a Trip ID wherever applicable.
- This enables complete trip-wise financial and vehicle-running history.

11. Trip Documents

- Support Delivery Challan, POD, Invoice, Vehicle Documents, and Other documents.
- Documents should store type, upload timestamp, linked trip, uploader, and file/photo metadata.

12. Digital ID & SOS

- Digital ID must be accessible from the driver app.
- SOS / accident reporting must provide a clear emergency action and record the incident for admin visibility.

13. Notifications

- Notify the driver for advance approval/rejection, expense approval/rejection, trip assignment/update, duty reminders, fuel submission, document requirements/approval, and settlement requirements.

14. Admin / Backend Visibility

- Admin should be able to see driver, vehicle, duty, odometer, KM, fuel, expenses, and advances.
- Driver detail page should include Profile, Duty, Fuel, Expenses, Advances, Documents, and Timeline.

15. Reports

- Daily/monthly reports should include duty, KM, fuel quantity, fuel cost, average KM/L, expenses, advances, settlements, and pending items.
- Reports should support driver-wise, vehicle-wise, and trip-wise views.

16. Photo & File Storage

- Store uploaded photos/files with file URL or storage key, file type, size, upload timestamp, uploader, and linked business record.
- Photos should be stored in durable backend/object storage rather than browser-only object URLs.

17. Audit Trail

- Track important create/update/approve/reject/settle actions with user, timestamp, record, old value where applicable, and new value.

18. Validation & Automation

- Use server-side validation for odometer, litres, cost, expense amount, and advance amount.
- Auto-capture timestamps; manual date/time editing should be restricted unless explicitly permitted by admin.
- Prevent duplicate submissions using request IDs/idempotency where appropriate.

19. Offline & Sync

- Driver app should support offline capture for key field operations where practical.
- Queue unsynced records locally and automatically sync when connectivity returns.
- Show sync status and handle conflicts without silently losing data.

20. Security & Permissions

- Use role-based access for Driver, Dispatcher/Admin, Finance, and other configured roles.
- Protect personal data, documents, financial records, and authentication credentials.
- APIs must verify that the authenticated driver can access or modify the requested records.

21. Suggested Database Entities

- drivers
- vehicles
- driver_vehicle_assignments
- trips
- duty_logs
- fuel_entries
- expenses
- advances
- documents
- transactions
- notifications

- audit_logs

22. End-to-End Driver Flows

- Morning: Login → Start Duty → Enter/scan starting odometer → Capture photo → Assigned trip visible.
- Fuel: Add Fuel → Odometer → Litres → Cost → Pump-meter photo → Submit → Backend saves entry.
- Expense: Add Expense → Category → Amount → Note → Receipt photo → Submit → Pending approval.
- Advance: Advance Request → Amount → Reason → Submit → Admin approval → Payment → Settlement.
- Evening: End Duty → Ending odometer → Photo → Total KM calculated → Duty history updated.

23. Development Priority

- Phase 1 – Core: Login, profile, vehicle, start/end duty, odometer, KM.
- Phase 2 – Financial: Fuel, photos, expenses, receipts, wallet, advances, approvals, settlement.
- Phase 3 – Trip: Trip management, documents, Digital ID, trip-wise records.
- Phase 4 – Reporting: Daily/monthly and driver/vehicle/trip reports.
- Phase 5 – Reliability: Offline mode, sync, audit logs, notifications, security hardening.

24. Production Goal

- The production system should provide a complete, traceable history for every driver, vehicle, and trip: duty → odometer → kilometres → fuel → fuel photos → expenses → receipts → advances → payments → documents.

Core Data Model – Quick Reference

Entity	Key purpose
drivers	Driver identity, contact, licence and status
vehicles	Vehicle master and configuration
driver_vehicle_assignments	Driver-to-vehicle assignment history
trips	Trip master and trip status
duty_logs	Start/end duty, odometer, KM, photos
fuel_entries	Fuel quantity, cost, odometer, pump photo
expenses	Expense category, amount, receipt and approval
advances	Advance requests, approvals, payments and settlement
documents	POD, challan, invoice and vehicle documents
transactions	Wallet/petty-cash ledger
notifications	Driver/admin alerts
audit_logs	Traceability of important changes

Note: This document is based on the Driver App prototype flows and the previously defined functional requirements. It is intended as a developer-facing specification; UI styling and exact API/database field names can be finalized during implementation.